Software Project Planning

Size estimation:

Size estimation in software engineering involves predicting the size of a software project in terms of lines of code, function points

Software size estimation is a crucial aspect of the software engineering process

Size estimation in software engineering provides a foundation for determining project timelines, effort required, and resource allocation.

Size estimation allows project managers to make informed decisions regarding project scope, budget, and staffing

Size estimation can be done using various techniques, including algorithmic methods, expert judgement, and machine learning approaches.

Cost estimation:

Cost estimation is a process of predict/estimate the approximate cost of the software project before the development start

Cost estimation in project management is the process of forecasting the cost and resources needed to complete a project within a defined scope. Cost estimation accounts for each element required for the project and calculates a total amount that determines a project's budget

Project Scheduling:

A project schedule indicates what needs to be done, which resources must be utilized, and when the project is due. It's a timetable that outlines start and end dates and milestones that must be met for the project to be completed on time

Project-task scheduling is a significant project planning activity. It comprises deciding which functions would be taken up when. To schedule the project plan, a software project manager wants to do the following:

1. Identify all the functions required to complete the project.
2. Break down large functions into small activities.
3. Determine the dependency among various activities.
4. Establish the most likely size for the time duration required to complete the activities.
5. Allocate resources to activities.
6. Plan the beginning and ending dates for different activities.

The constructive cost model (COCOMO)

Boehm proposed COCOMO (Constructive Cost Estimation Model) in 1981.COCOMO is one of the most generally used software estimation models in the world. COCOMO predicts the efforts and schedule of a software product based on the size of the software

COCOMO 1 model provides estimates of schedule and effort. It uses the total number of code lines delivered to produce estimations. Three development modes define the exponent of COCOMO 1's effort equation. The COCOMO 1 model is allocated ***15 cost drivers***. It is important in the waterfall models of SDLC. The software projects are divided mainly into ***3 types*** to estimate the effort accurately. These are as follows:**In COCOMO, projects are categorized into three types:**

1. Organic
2. Semidetached
3. Embedded

1. Organic Projects

Organic projects are not particularly large in size, and it only contains ***50 KDLOC (a kilo of delivered lines of codes or less)***. It necessitates a pre-experienced team and a deep knowledge of the software project. These projects are simple to create and have no time constraints. Some examples of organic projects are business systems, inventory management systems, payroll management systems, and many others.

2. Embedded Projects

These types of projects are highly sophisticated and contain ***300 KDLOC*** or more. The team necessary to complete these projects does not require being highly experienced and new developers can also readily participate in these initiatives. However, when developing these projects, users must adhere to strict constraints (hardware, software, people, and deadlines) and must meet the user's stringent requirements. These projects include software systems that are utilized in avionics and military technology.

3. Semi-detached Projects

This category falls between organic and embedded initiatives. Similarly, the complexity of these COCOMO projects falls somewhere between embedded and organic projects, where less than 300,000 lines of code may be delivered. An averagely experienced user and a medium timeframe are required to make the goods. These projects could include the design of OS, databases, compilers, and other things.

1. **Effort:** Amount of labor that will be required to complete a task. It is measured in person-months units.
2. **Schedule:** This simply means the amount of time required for the completion of the job, which is, of course, proportional to the effort put in. It is measured in the units of time such as weeks, and months

Types of COCOMO Model

The different types of COCOMO models define the depth of cost estimation is required for the project. It depends on the software manager, what type of model do they choose.

- Basic Model
- Intermediate Model
- Detailed Model

Basic Model – This model is based on rough calculations thus, there is very limited accuracy. The whole model is based on only lines of source code to estimate the calculation and other factors are neglected.

Intermediate Model – The intermediate model dives deeper and includes more factors such as cost drivers into the calculation. This enhances the accuracy of estimation. The cost driver includes attributes like reliability, capability, and experience.

Detailed Model – The detailed model or the complete model includes all the factors of the both-the basic model and the intermediate model. In the detailed model for each cost driver property, various effort multipliers are used.

Calculation

The effort and the time taken to complete both are calculated through predefined equations

The formula used is-

Effort(E)= a*(KLOC)^b PM(Person month)

Time = c(E)d

Person Required = effort/time

Where,
KLOC = size of the product i.e. no. of lines of code.
a, b, c, d are constants and have different values for different models.
Time is the time required to develop the product and the unit is months.
And the effort is the total effort calculated that will be required to complete the work calculated in PM (Person Months).

If the constants are replaced by actual values, then the equations will be

Organic-
E = 2.4(KLOC) 1.05 PM
T = 2.5(E) 0.38 Months

Semi-detached-
E= 3.0(KLOC) 1.12 PM
T = 2.5(E) 0.35 Months

Embedded-
E= 3.6(KLOC) 1.20 PM
T= 2.5(E) 0.32 Months

	A	B	C	D
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

What is COCOMO 2?

COCOMO 2 is another cost-estimating methodology utilized to calculate the cost of software development. It is a modified version of the original ***COCOMO*** that was developed at the ***University of Southern California***. It was mainly designed to resolve the shortcomings of the COCOMO 1 model. Its primary goal is to offer methodologies, quantitative analytic structure, and tools. It computes the total development time and effort that is based on the estimates of all individual subsystems

COCOMO 2 estimation models

There are mainly 4 types of COCOMO 2 estimation models. These models are as follows:

1. **Application Composition Model**

   It is intended for usage with reusable components to create prototype development estimates and operates on the basis of object points. It is more suited to prototype system development.

2. **Early Design Model**
   After gathering requirements, the model is utilized during the system design phase. It generates estimates based on function points, which are subsequently converted into numerous lines of source code. The estimates at this level are based on the basic algorithm model formula. You may utilize the following formula:

res	COCOMO 1	COCOMO 2
Full Forms	COCOMO 1 is an abbreviation for Constructive Cost Model 1.	COCOMO 2 is an abbreviation for Constructive Cost Model 2.
Utilization Models	It is utilized in the waterfall model of SDLC.	It is useful in quick development, non-sequential, and reuses software models.
Basic	It is based on the linear reuse formula.	It is based on the non-linear formula.
Estimation Precision	It offers estimates of the effort and timeline.	It offers estimates that are one standard deviation from the most common estimate.
Size of program statements	The program's size is indicated in code lines.	It has extra factors for expressing software size, including lines of code, object points, and function points.
Model framework	The requirements given to the software come first in the development process.	It utilizes the spiral type of development.
Effort equation's exponent	The 3 development methods define the exponent of the effort equation.	The 5 scale methods define the exponent of the effort equation.
Sub models	It has 3 submodels.	It has 4 submodels.
Cost Drivers	It utilizes 15 cost drivers.	It utilizes 17 cost drivers.

Software risk management

Risk Management is a systematic process of recognizing, evaluating, and handling threats or risks that have an effect on the finances, capital, and overall operations of an organization. These risks can come from different areas, such as financial instability, legal issues, errors in strategic planning, accidents, and natural disasters.

The main goal of risk management is to predict possible risks and find solutions to deal with them successfully.

There are three main classifications of risks which can affect a software project

1. Project risks
2. Technical risks
3. Business risks

. Project risks: Project risks concern differ forms of budgetary, schedule, personnel, resource, and customer-related problems

. Technical risks: Technical risks concern potential method, implementation, interfacing, testing, and maintenance issue

Business risks: This type of risks contain risks of building an excellent product that no one need, losing budgetary or personnel commitments, etc

The risk management process

Risk management is a sequence of steps that help a software team to understand, analyze, and manage uncertainty. Risk management process consists of

- Risks Identification.
- **Risk analysis**
- Risks Planning.
- Risk Monitoring